

SupportPilot

Milestone 2 — Complete Feature List

Knowledge Retrieval & Resolution | Based on the uploaded Milestone 2 Data Models Specification

1. Milestone 1 Integration

- Builds on the 11 Milestone 1 collections.
- tickets is extended with resolution-related fields and a new PENDING_REVIEW state.
- category_taxonomy is extended with knowledge-base coverage fields.
- Existing collections remain compatible; the specification states no migration is required.

2. Knowledge Base

- Manage authored knowledge articles.
- Publishing workflow: DRAFT, PUBLISHED, ARCHIVED.
- Categories, sub-categories, tags and Markdown content.
- Track source system, source URL, indexing state, author and reviewer.
- Department visibility and internal-only access controls.

3. Article Versioning

- Immutable, insert-only version history.
- Store article ID, version, title, content, change note, changer and timestamp.
- Unique version numbering per article.

4. Article Chunking

- Split articles into retrieval chunks.
- Store article relationship, chunk order, title/status/category, heading path, content and token count.
- Store 1024-dimensional embeddings with embedding model metadata.
- Denormalized parent fields support filtering directly inside the indexed collection.

5. Document Ingestion

- Support BULK_UPLOAD, CONFLUENCE_SYNC and REINDEX jobs.
- Track progress: documents processed, articles created/updated, failures, chunks created and embedded.
- Per-document error tracking so one failed document does not fail an entire batch.

6. Vector Search

- Atlas Vector Search on article_chunks.
- Index: kb_vector_index.
- Embedding path: embedding.

- 1024 dimensions, cosine similarity.
- Filter by article_status, category and embedding_model.

7. Text Search

- Atlas Search text index: kb_text_index.
- Search content, heading_path and article_title.
- Filter for published articles and use weighted heading/title matching.

8. Hybrid Retrieval

- Run vector and keyword searches concurrently.
- Retrieve candidates from both paths.
- Fuse results, then rerank the top candidates.
- Use the resulting evidence to support ticket resolution.

9. AI Resolution Generation

- Generate a ticket resolution with sufficient_context indicator.
- Create a summary and ordered resolution steps.
- Store confidence and confidence components.
- Record queries used, chunks retrieved, model, prompt version, token usage and latency.
- Resolution lifecycle: DRAFT, SENT, EDITED_SENT, REJECTED.

10. Citation Guardrails

- Each generated step contains source references to knowledge articles and chunks.
- Store independent flattened citations for analytics.
- Uncited steps are dropped before agent review.
- Record steps_dropped and dropped_details for quality and hallucination monitoring.

11. Resolution Review Workflow

- Generated resolution begins as DRAFT.
- Agent review can accept/send, edit and send, or reject.
- Ticket state transition: AI_PROCESSING → PENDING_REVIEW.
- After review: PENDING_REVIEW → IN_PROGRESS on accept, or → ESCALATED on reject.

12. Human Approval & Destructive Actions

- Resolution steps can be marked requires_approval.
- Destructive actions are marked is_destructive.
- Store destructive_reason when applicable.

13. Resolution Feedback

- Requester feedback: helpful or not helpful.

- Optional feedback comment.
- Track whether the ticket was resolved without escalation.

14. Knowledge Gap Management

- When context is insufficient, record and aggregate knowledge gaps.
- Group by category and sub_category.
- Track occurrence_count, recent ticket IDs, sample queries and sample subjects.
- Queue statuses: OPEN, WRITING, RESOLVED, WONT_FIX.
- Prioritize OPEN gaps by occurrence count.

15. Retrieval Logging

- Store extracted queries and retrieval evidence.
- Log vector hits, keyword hits, fused results and reranked results.
- Track query, vector, keyword, fusion, rerank and total latency.
- Record degraded retrieval components.
- TTL retention: 30 days.

16. Ticket Collection Changes

- Add has_resolution.
- Add resolution_status: DRAFT, SENT, EDITED_SENT, REJECTED, NONE.
- Add latest_response_id.
- Add PENDING_REVIEW to the ticket status state machine.

17. Category Knowledge Coverage

- Add kb_article_count.
- Add kb_coverage: GOOD, THIN, MISSING.
- Add last_gap_seen_at.
- Expose coverage information to make content gaps visible in administration.

18. Transactions

- Publish article: update article, insert version, replace chunks atomically.
- Archive article: update article and chunk status in one transaction.
- Store resolution: insert response, insert citations, update ticket flags/status.
- Record refusal: store insufficient-context response and upsert kb_gaps.
- Accept and send: update response, create AI ticket comment, transition ticket.
- Bulk ingestion: per-document transaction, not per-batch.

19. Indexes

- knowledge_articles: unique slug; status/category; stale-index sweep; updated_at.

- article_versions: article/version history and unique article_id + version.
- article_chunks: vector index, text index, article/chunk order, embedding model.
- ingestion_jobs: status/started_at and triggered_by_id.
- ticket_responses: ticket/date, status/date, sufficient_context/date, steps_dropped.
- response_citations: response, article usage analytics, ticket.
- resolution_feedback: response and helpfulness/date.
- kb_gaps: unique category/sub_category and status/occurrence queue.
- retrieval_logs: ticket, TTL created_at, latency.total_ms.
- tickets: resolution_status and created_at.

20. Validation Requirements

- Create all 9 new collections with strict \$jsonSchema validation and validationAction: error.
- article_chunks requires article_id, chunk_index, content, embedding, embedding_model and article_status.
- chunk_index minimum 0.
- content minimum length 20.
- token_count minimum 50 and maximum 1200.
- article_status enum: DRAFT, PUBLISHED, ARCHIVED.
- embedding must contain exactly 1024 double values using minItems and maxItems.
- Confirm both kb_vector_index and kb_text_index are queryable.
- Poll for Atlas index availability during ingestion verification.
- Test archive/publish atomicity, concurrent hybrid retrieval, citation dropping, KB-gap upsert, TTL behavior, stale-index sweep and index usage.

Milestone 2 Collections

#	Collection	Type / Purpose
1	knowledge_articles	Knowledge-base article with publishing workflow
2	article_versions	Immutable article version history
3	article_chunks	Vector and text search retrieval target
4	ingestion_jobs	Bulk import and per-document failure tracking
5	ticket_responses	Generated ticket resolutions with embedded steps
6	response_citations	Independently queryable resolved citations
7	resolution_feedback	Helpful/not-helpful requester feedback
8	kb_gaps	Aggregated knowledge-gap work queue
9	retrieval_logs	Diagnostic retrieval evidence and latency logs

Key Workflow States

Area	States / Flow
Article	DRAFT → PUBLISHED → ARCHIVED
Resolution	DRAFT → SENT / EDITED_SENT / REJECTED
Ticket Review	AI_PROCESSING → PENDING_REVIEW → IN_PROGRESS (accept) or ESCALATED (reject)
KB Gap	OPEN → WRITING → RESOLVED or WONT_FIX
Knowledge Coverage	GOOD THIN MISSING

Scope summary: Milestone 2 adds the knowledge-retrieval and resolution data layer on top of Milestone 1, combining article management, ingestion, chunking, Atlas Vector Search, Atlas Search, hybrid retrieval, generated resolutions, citations, feedback, knowledge-gap tracking, diagnostics, transactional consistency, indexes and schema validation.